

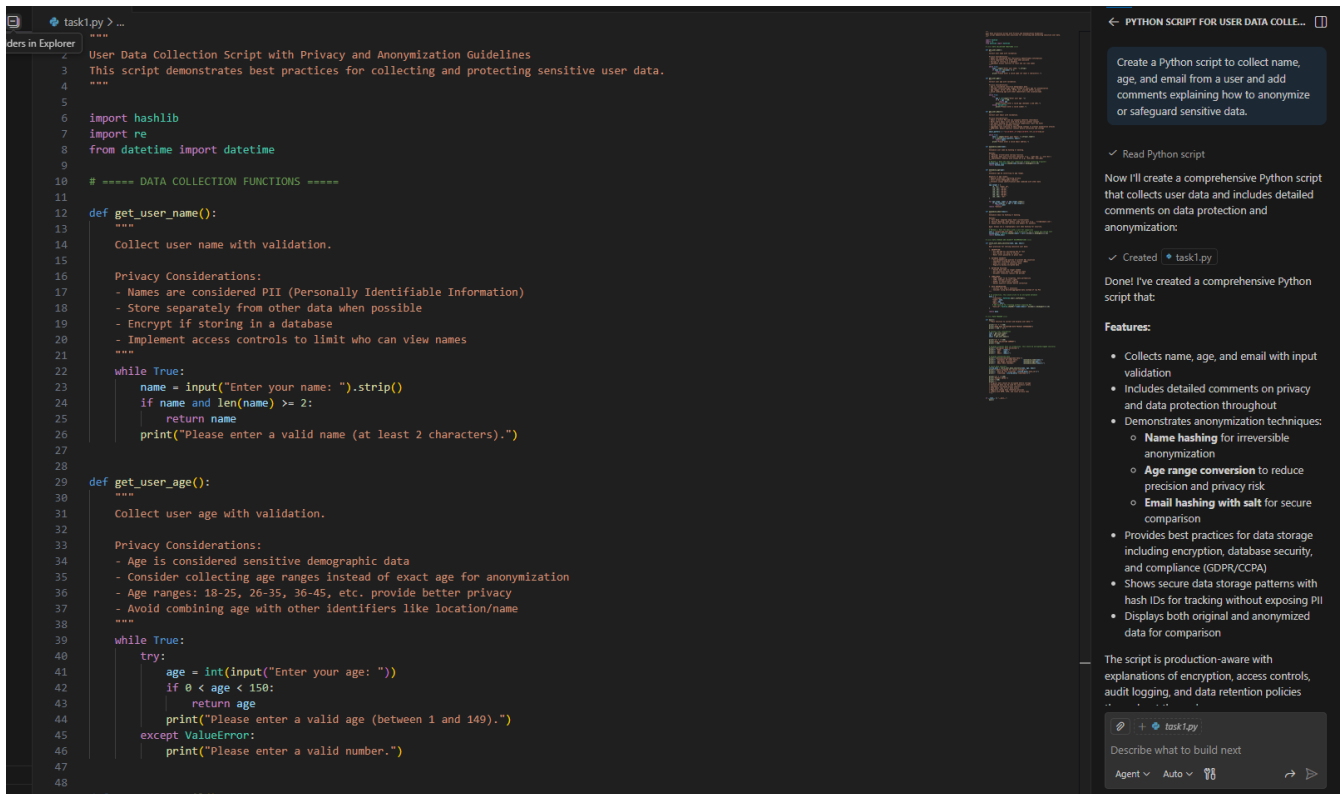
Assignment: 5.4

Name: S. Manohar

Hno: 2303A51982

Task Description #1:

- Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.



```
task1.py > ...
User Data Collection Script with Privacy and Anonymization Guidelines
This script demonstrates best practices for collecting and protecting sensitive user data.
'''
5
import hashlib
import re
from datetime import datetime
9
# ===== DATA COLLECTION FUNCTIONS =====
11
def get_user_name():
'''
13
Collect user name with validation.
15
Privacy Considerations:
17
- Names are considered PII (Personally Identifiable Information)
18
- Store separately from other data when possible
19
- Encrypt if storing in a database
20
- Implement access controls to limit who can view names
21
'''
22
while True:
23
    name = input("Enter your name: ").strip()
24
    if name and len(name) >= 2:
25
        return name
26
    print("Please enter a valid name (at least 2 characters).")
27
28
def get_user_age():
'''
30
Collect user age with validation.
32
Privacy Considerations:
34
- Age is considered sensitive demographic data
35
- Consider collecting age ranges instead of exact age for anonymization
36
- Age ranges: 18-25, 26-35, 36-45, etc. provide better privacy
37
- Avoid combining age with other identifiers like location/name
38
'''
39
while True:
40
    try:
41
        age = int(input("Enter your age: "))
42
        if 0 < age < 150:
43
            return age
44
        print("Please enter a valid age (between 1 and 149).")
45
    except ValueError:
46
        print("Please enter a valid number.")
47
48
def get_user_email():
'''
49
Collect user email with validation.
51
Privacy Considerations:
53
- Email addresses are considered PII
54
- Use hashed email addresses for tracking and analytics
55
- Implement email verification process
56
- Store encrypted email addresses
57
- Implement access controls to limit who can view email addresses
58
'''
59
while True:
60
    email = input("Enter your email: ").strip()
61
    if email and re.match(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", email):
62
        return email
63
    print("Please enter a valid email address.")
64
65
def main():
'''
67
Main function to collect user data.
69
'''
70
print("Welcome to the User Data Collection Script")
71
print("This script will collect your name, age, and email address.")
72
print("Your data will be stored securely and anonymized where possible.")
73
print("You can exit the script at any time by pressing Ctrl+C.")
74
75
# Collect user data
76
name = get_user_name()
77
age = get_user_age()
78
email = get_user_email()
79
80
# Anonymization and Protection
81
# Hash the email address for anonymization
82
hashed_email = hashlib.sha256(email.encode()).hexdigest()
83
84
# Store data in a secure database
85
# (This is a placeholder for database storage logic)
86
87
# Display collected data
88
print(f"User Name: {name}")
89
print(f>User Age: {age}")
90
print(f>User Email: {hashed_email}")
91
92
# End of Script
93
'''
94
'''
95
if __name__ == "__main__":
96
    main()
97
'''
98
'''
99
'''
100
'''
101
'''
102
'''
103
'''
104
'''
105
'''
106
'''
107
'''
108
'''
109
'''
110
'''
111
'''
112
'''
113
'''
114
'''
115
'''
116
'''
117
'''
118
'''
119
'''
120
'''
121
'''
122
'''
123
'''
124
'''
125
'''
126
'''
127
'''
128
'''
129
'''
130
'''
131
'''
132
'''
133
'''
134
'''
135
'''
136
'''
137
'''
138
'''
139
'''
140
'''
141
'''
142
'''
143
'''
144
'''
145
'''
146
'''
147
'''
148
'''
149
'''
150
'''
151
'''
152
'''
153
'''
154
'''
155
'''
156
'''
157
'''
158
'''
159
'''
160
'''
161
'''
162
'''
163
'''
164
'''
165
'''
166
'''
167
'''
168
'''
169
'''
170
'''
171
'''
172
'''
173
'''
174
'''
175
'''
176
'''
177
'''
178
'''
179
'''
180
'''
181
'''
182
'''
183
'''
184
'''
185
'''
186
'''
187
'''
188
'''
189
'''
190
'''
191
'''
192
'''
193
'''
194
'''
195
'''
196
'''
197
'''
198
'''
199
'''
200
'''
201
'''
202
'''
203
'''
204
'''
205
'''
206
'''
207
'''
208
'''
209
'''
210
'''
211
'''
212
'''
213
'''
214
'''
215
'''
216
'''
217
'''
218
'''
219
'''
220
'''
221
'''
222
'''
223
'''
224
'''
225
'''
226
'''
227
'''
228
'''
229
'''
230
'''
231
'''
232
'''
233
'''
234
'''
235
'''
236
'''
237
'''
238
'''
239
'''
240
'''
241
'''
242
'''
243
'''
244
'''
245
'''
246
'''
247
'''
248
'''
249
'''
250
'''
251
'''
252
'''
253
'''
254
'''
255
'''
256
'''
257
'''
258
'''
259
'''
260
'''
261
'''
262
'''
263
'''
264
'''
265
'''
266
'''
267
'''
268
'''
269
'''
270
'''
271
'''
272
'''
273
'''
274
'''
275
'''
276
'''
277
'''
278
'''
279
'''
280
'''
281
'''
282
'''
283
'''
284
'''
285
'''
286
'''
287
'''
288
'''
289
'''
290
'''
291
'''
292
'''
293
'''
294
'''
295
'''
296
'''
297
'''
298
'''
299
'''
300
'''
301
'''
302
'''
303
'''
304
'''
305
'''
306
'''
307
'''
308
'''
309
'''
310
'''
311
'''
312
'''
313
'''
314
'''
315
'''
316
'''
317
'''
318
'''
319
'''
320
'''
321
'''
322
'''
323
'''
324
'''
325
'''
326
'''
327
'''
328
'''
329
'''
330
'''
331
'''
332
'''
333
'''
334
'''
335
'''
336
'''
337
'''
338
'''
339
'''
340
'''
341
'''
342
'''
343
'''
344
'''
345
'''
346
'''
347
'''
348
'''
349
'''
350
'''
351
'''
352
'''
353
'''
354
'''
355
'''
356
'''
357
'''
358
'''
359
'''
360
'''
361
'''
362
'''
363
'''
364
'''
365
'''
366
'''
367
'''
368
'''
369
'''
370
'''
371
'''
372
'''
373
'''
374
'''
375
'''
376
'''
377
'''
378
'''
379
'''
380
'''
381
'''
382
'''
383
'''
384
'''
385
'''
386
'''
387
'''
388
'''
389
'''
390
'''
391
'''
392
'''
393
'''
394
'''
395
'''
396
'''
397
'''
398
'''
399
'''
400
'''
401
'''
402
'''
403
'''
404
'''
405
'''
406
'''
407
'''
408
'''
409
'''
410
'''
411
'''
412
'''
413
'''
414
'''
415
'''
416
'''
417
'''
418
'''
419
'''
420
'''
421
'''
422
'''
423
'''
424
'''
425
'''
426
'''
427
'''
428
'''
429
'''
430
'''
431
'''
432
'''
433
'''
434
'''
435
'''
436
'''
437
'''
438
'''
439
'''
440
'''
441
'''
442
'''
443
'''
444
'''
445
'''
446
'''
447
'''
448
'''
449
'''
450
'''
451
'''
452
'''
453
'''
454
'''
455
'''
456
'''
457
'''
458
'''
459
'''
460
'''
461
'''
462
'''
463
'''
464
'''
465
'''
466
'''
467
'''
468
'''
469
'''
470
'''
471
'''
472
'''
473
'''
474
'''
475
'''
476
'''
477
'''
478
'''
479
'''
480
'''
481
'''
482
'''
483
'''
484
'''
485
'''
486
'''
487
'''
488
'''
489
'''
490
'''
491
'''
492
'''
493
'''
494
'''
495
'''
496
'''
497
'''
498
'''
499
'''
500
'''
501
'''
502
'''
503
'''
504
'''
505
'''
506
'''
507
'''
508
'''
509
'''
510
'''
511
'''
512
'''
513
'''
514
'''
515
'''
516
'''
517
'''
518
'''
519
'''
520
'''
521
'''
522
'''
523
'''
524
'''
525
'''
526
'''
527
'''
528
'''
529
'''
530
'''
531
'''
532
'''
533
'''
534
'''
535
'''
536
'''
537
'''
538
'''
539
'''
540
'''
541
'''
542
'''
543
'''
544
'''
545
'''
546
'''
547
'''
548
'''
549
'''
550
'''
551
'''
552
'''
553
'''
554
'''
555
'''
556
'''
557
'''
558
'''
559
'''
560
'''
561
'''
562
'''
563
'''
564
'''
565
'''
566
'''
567
'''
568
'''
569
'''
570
'''
571
'''
572
'''
573
'''
574
'''
575
'''
576
'''
577
'''
578
'''
579
'''
580
'''
581
'''
582
'''
583
'''
584
'''
585
'''
586
'''
587
'''
588
'''
589
'''
590
'''
591
'''
592
'''
593
'''
594
'''
595
'''
596
'''
597
'''
598
'''
599
'''
600
'''
601
'''
602
'''
603
'''
604
'''
605
'''
606
'''
607
'''
608
'''
609
'''
610
'''
611
'''
612
'''
613
'''
614
'''
615
'''
616
'''
617
'''
618
'''
619
'''
620
'''
621
'''
622
'''
623
'''
624
'''
625
'''
626
'''
627
'''
628
'''
629
'''
630
'''
631
'''
632
'''
633
'''
634
'''
635
'''
636
'''
637
'''
638
'''
639
'''
640
'''
641
'''
642
'''
643
'''
644
'''
645
'''
646
'''
647
'''
648
'''
649
'''
650
'''
651
'''
652
'''
653
'''
654
'''
655
'''
656
'''
657
'''
658
'''
659
'''
660
'''
661
'''
662
'''
663
'''
664
'''
665
'''
666
'''
667
'''
668
'''
669
'''
670
'''
671
'''
672
'''
673
'''
674
'''
675
'''
676
'''
677
'''
678
'''
679
'''
680
'''
681
'''
682
'''
683
'''
684
'''
685
'''
686
'''
687
'''
688
'''
689
'''
690
'''
691
'''
692
'''
693
'''
694
'''
695
'''
696
'''
697
'''
698
'''
699
'''
700
'''
701
'''
702
'''
703
'''
704
'''
705
'''
706
'''
707
'''
708
'''
709
'''
710
'''
711
'''
712
'''
713
'''
714
'''
715
'''
716
'''
717
'''
718
'''
719
'''
720
'''
721
'''
722
'''
723
'''
724
'''
725
'''
726
'''
727
'''
728
'''
729
'''
730
'''
731
'''
732
'''
733
'''
734
'''
735
'''
736
'''
737
'''
738
'''
739
'''
740
'''
741
'''
742
'''
743
'''
744
'''
745
'''
746
'''
747
'''
748
'''
749
'''
750
'''
751
'''
752
'''
753
'''
754
'''
755
'''
756
'''
757
'''
758
'''
759
'''
760
'''
761
'''
762
'''
763
'''
764
'''
765
'''
766
'''
767
'''
768
'''
769
'''
770
'''
771
'''
772
'''
773
'''
774
'''
775
'''
776
'''
777
'''
778
'''
779
'''
780
'''
781
'''
782
'''
783
'''
784
'''
785
'''
786
'''
787
'''
788
'''
789
'''
790
'''
791
'''
792
'''
793
'''
794
'''
795
'''
796
'''
797
'''
798
'''
799
'''
800
'''
801
'''
802
'''
803
'''
804
'''
805
'''
806
'''
807
'''
808
'''
809
'''
810
'''
811
'''
812
'''
813
'''
814
'''
815
'''
816
'''
817
'''
818
'''
819
'''
820
'''
821
'''
822
'''
823
'''
824
'''
825
'''
826
'''
827
'''
828
'''
829
'''
830
'''
831
'''
832
'''
833
'''
834
'''
835
'''
836
'''
837
'''
838
'''
839
'''
840
'''
841
'''
842
'''
843
'''
844
'''
845
'''
846
'''
847
'''
848
'''
849
'''
850
'''
851
'''
852
'''
853
'''
854
'''
855
'''
856
'''
857
'''
858
'''
859
'''
860
'''
861
'''
862
'''
863
'''
864
'''
865
'''
866
'''
867
'''
868
'''
869
'''
870
'''
871
'''
872
'''
873
'''
874
'''
875
'''
876
'''
877
'''
878
'''
879
'''
880
'''
881
'''
882
'''
883
'''
884
'''
885
'''
886
'''
887
'''
888
'''
889
'''
890
'''
891
'''
892
'''
893
'''
894
'''
895
'''
896
'''
897
'''
898
'''
899
'''
900
'''
901
'''
902
'''
903
'''
904
'''
905
'''
906
'''
907
'''
908
'''
909
'''
910
'''
911
'''
912
'''
913
'''
914
'''
915
'''
916
'''
917
'''
918
'''
919
'''
920
'''
921
'''
922
'''
923
'''
924
'''
925
'''
926
'''
927
'''
928
'''
929
'''
930
'''
931
'''
932
'''
933
'''
934
'''
935
'''
936
'''
937
'''
938
'''
939
'''
940
'''
941
'''
942
'''
943
'''
944
'''
945
'''
946
'''
947
'''
948
'''
949
'''
950
'''
951
'''
952
'''
953
'''
954
'''
955
'''
956
'''
957
'''
958
'''
959
'''
960
'''
961
'''
962
'''
963
'''
964
'''
965
'''
966
'''
967
'''
968
'''
969
'''
970
'''
971
'''
972
'''
973
'''
974
'''
975
'''
976
'''
977
'''
978
'''
979
'''
980
'''
981
'''
982
'''
983
'''
984
'''
985
'''
986
'''
987
'''
988
'''
989
'''
990
'''
991
'''
992
'''
993
'''
994
'''
995
'''
996
'''
997
'''
998
'''
999
'''
1000
'''
1001
'''
1002
'''
1003
'''
1004
'''
1005
'''
1006
'''
1007
'''
1008
'''
1009
'''
1010
'''
1011
'''
1012
'''
1013
'''
1014
'''
1015
'''
1016
'''
1017
'''
1018
'''
1019
'''
1020
'''
1021
'''
1022
'''
1023
'''
1024
'''
1025
'''
1026
'''
1027
'''
1028
'''
1029
'''
1030
'''
1031
'''
1032
'''
1033
'''
1034
'''
1035
'''
1036
'''
1037
'''
1038
'''
1039
'''
1040
'''
1041
'''
1042
'''
1043
'''
1044
'''
1045
'''
1046
'''
1047
'''
1048
'''
1049
'''
1050
'''
1051
'''
1052
'''
1053
'''
1054
'''
1055
'''
1056
'''
1057
'''
1058
'''
1059
'''
1060
'''
1061
'''
1062
'''
1063
'''
1064
'''
1065
'''
1066
'''
1067
'''
1068
'''
1069
'''
1070
'''
1071
'''
1072
'''
1073
'''
1074
'''
1075
'''
1076
'''
1077
'''
1078
'''
1079
'''
1080
'''
1081
'''
1082
'''
1083
'''
1084
'''
1085
'''
1086
'''
1087
'''
1088
'''
1089
'''
1090
'''
1091
'''
1092
'''
1093
'''
1094
'''
1095
'''
1096
'''
1097
'''
1098
'''
1099
'''
1100
'''
1101
'''
1102
'''
1103
'''
1104
'''
1105
'''
1106
'''
1107
'''
1108
'''
1109
'''
1110
'''
1111
'''
1112
'''
1113
'''
1114
'''
1115
'''
1116
'''
1117
'''
1118
'''
1119
'''
1120
'''
1121
'''
1122
'''
1123
'''
1124
'''
1125
'''
1126
'''
1127
'''
1128
'''
1129
'''
1130
'''
1131
'''
1132
'''
1133
'''
1134
'''
1135
'''
1136
'''
1137
'''
1138
'''
1139
'''
1140
'''
1141
'''
1142
'''
1143
'''
1144
'''
1145
'''
1146
'''
1147
'''
1148
'''
1149
'''
1150
'''
1151
'''
1152
'''
1153
'''
1154
'''
1155
'''
1156
'''
1157
'''
1158
'''
1159
'''
1160
'''
1161
'''
1162
'''
1163
'''
1164
'''
1165
'''
1166
'''
1167
'''
1168
'''
1169
'''
1170
'''
1171
'''
1172
'''
1173
'''
1174
'''
1175
'''
1176
'''
1177
'''
1178
'''
1179
'''
1180
'''
1181
'''
1182
'''
1183
'''
1184
'''
1185
'''
1186
'''
1187
'''
1188
'''
1189
'''
1190
'''
1191
'''
1192
'''
1193
'''
1194
'''
1195
'''
1196
'''
1197
'''
1198
'''
1199
'''
1200
'''
1201
'''
1202
'''
1203
'''
1204
'''
1205
'''
1206
'''
1207
'''
1208
'''
1209
'''
1210
'''
1211
'''
1212
'''
1213
'''
1214
'''
1215
'''
1216
'''
1217
'''
1218
'''
1219
'''
1220
'''
1221
'''
1222
'''
1223
'''
1224
'''
1225
'''
1226
'''
1227
'''
1228
'''
1229
'''
1230
'''
1231
'''
1232
'''
1233
'''
1234
'''
1235
'''
1236
'''
1237
'''
1238
'''
1239
'''
1240
'''
1241
'''
1242
'''
1243
'''
1244
'''
1245
'''
1246
'''
1247
'''
1248
'''
1249
'''
1250
'''
1251
'''
1252
'''
1253
'''
1254
'''
1255
'''
1256
'''
1257
'''
1258
'''
1259
'''
1260
'''
1261
'''
1262
'''
1263
'''
1264
'''
1265
'''
1266
'''
1267
'''
1268
'''
1269
'''
1270
'''
1271
'''
1272
'''
1273
'''
1274
'''
1275
'''
1276
'''
1277
'''
1278
'''
1279
'''
1280
'''
1281
'''
1282
'''
1283
'''
1284
'''
1285
'''
1286
'''
1287
'''
1288
'''
1289
'''
1290
'''
1291
'''
1292
'''
1293
'''
1294
'''
1295
'''
1296
'''
1297
'''
1298
'''
1299
'''
1300
'''
1301
'''
1302
'''
1303
'''
1304
'''
1305
'''
1306
'''
1307
'''
1308
'''
1309
'''
1310
'''
1311
'''
1312
'''
1313
'''
1314
'''
1315
'''
1316
'''
1317
'''
1318
'''
1319
'''
1320
'''
1321
'''
1322
'''
1323
'''
1324
'''
1325
'''
1326
'''
1327
'''
1328
'''
1329
'''
1330
'''
1331
'''
1332
'''
1333
'''
1334
'''
1335
'''
1336
'''
1337
'''
1338
'''
1339
'''
1340
'''
1341
'''
1342
'''
1343
'''
1344
'''
1345
'''
1346
'''
1347
'''
1348
'''
1349
'''
1350
'''
1351
'''
1352
'''
1353
'''
1354
'''
1355
'''
1356
'''
1357
'''
1358
'''
1359
'''
1360
'''
1361
'''
1362
'''
1363
'''
1364
'''
1365
'''
1366
'''
1367
'''
1368
'''
1369
'''
1370
'''
1371
'''
1372
'''
1373
'''
1374
'''
1375
'''
1376
'''
1377
'''
1378
'''
1379
'''
1380
'''
1381
'''
1382
'''
1383
'''
1384
'''
1385
'''
1386
'''
1387
'''
1388
'''
1389
'''
1390
'''
1391
'''
1392
'''
1393
'''
1394
'''
1395
'''
1396
'''
1397
'''
1398
'''
1399
'''
1400
'''
1401
'''
1402
'''
1403
'''
1404
'''
1405
'''
1406
'''
1407
'''
1408
'''
1409
'''
1410
'''
1411
'''
1412
'''
1413
'''
1414
'''
1415
'''
1416
'''
1417
'''
1418
'''
1419
'''
1420
'''
1421
'''
1422
'''
1423
'''
1424
'''
1425
'''
1426
'''
1427
'''
1428
'''
1429
'''
1430
'''
1431
'''
1432
'''
1433
'''
1434
'''
1435
'''
1436
'''
1437
'''
1438
'''
1439
'''
1440
'''
1441
'''
1442
'''
1443
'''
1444
'''
1445
'''
1446
'''
1447
'''
1448
'''
1449
'''
1450
'''
1451
'''
1452
'''
1453
'''
1454
'''
1455
'''
1456
'''
1457
'''
1458
'''
1459
'''
1460
'''
1461
'''
1462
'''
1463
'''
1464
'''
1465
'''
1466
'''
1467
'''
1468
'''
1469
'''
1470
'''
1471
'''
1472
'''
1473
'''
1474
'''
1475
'''
1476
'''
1477
'''
1478
'''
1479
'''
1480
'''
1481
'''
1482
'''
1483
'''
1484
'''
1485
'''
1486
'''
1487
'''
1488
'''
1489
'''
1490
'''
1491
'''
1492
'''
1493
'''
1494
'''
1495
'''
1496
'''
1497
'''
1498
'''
1499
'''
1500
'''
1501
'''
1502
'''
1503
'''
1504
'''
1505
'''
1506
'''
1507
'''
1508
'''
1509
'''
1510
'''
1511
'''
1512
'''
1513
'''
1514
'''
1515
'''
1516
'''
1517
'''
1518
'''
1519
'''
1520
'''
1521
'''
1522
'''
1523
'''
1524
'''
1525
'''
1526
'''
1527
'''
1528
'''
1529
'''
1530
'''
1531
'''
1532
'''
1533
'''
1534
'''
1535
'''
1536
'''
1537
'''
1538
'''
1539
'''
1540
'''
1541
'''
1542
'''
1543
'''
1544
'''
1545
'''
1546
'''
1547
'''
1548
'''
1549
'''
1550
'''
1551
'''
1552
'''
1553
'''
1554
'''
1555
'''
1556
'''
1557
'''
1558
'''
1559
'''
1560
'''
1561
'''
1562
'''
1563
'''
1564
'''
1565
'''
1566
'''
1567
'''
1568
'''
1569
'''
1570
'''
1571
'''
1572
'''
1573
'''
1574
'''
1575
'''
1576
'''
1577
'''
1578
'''
1579
'''
1580
'''
1581
'''
1582
'''
1583
'''
1584
'''
1585
'''
1586
'''
1587
'''
1588
'''
1589
'''
1590
'''
1591
'''
1592
'''
1593
'''
1594
'''
1595
'''
1596
'''
1597
'''
1598
'''
1599
'''
1600
'''
1601
'''
1602
'''
1603
'''
1604
'''
1605
'''
1606
'''
1607
'''
1608
'''
1609
'''
1610
'''
1611
'''
1612
'''
1613
'''
1614
'''
1615
'''
1616
'''
1617
'''
1618
'''
1619
'''
1620
'''
1621
'''
1622
'''
1623
'''
1624
'''
1625
'''
1626
'''
1627
'''
1628
'''
1629
'''
1630
'''
1631
'''
1632
'''
1633
'''
1634
'''
1635
'''
1636
'''
1637
'''
1638
'''
1639
'''
1640
'''
1641
'''
1642
'''
1643
'''
1644
'''
1645
'''
1646
'''
1647
'''
1648
'''
1649
'''
1650
'''
1651
'''
1652
'''
1653
'''
1654
'''
1655
'''
1656
'''
1657
'''
1658
'''
1659
'''
1660
'''
1661
'''
1662
'''
1663
'''
1664
'''
1665
'''
1666
'''
1667
'''
1668
'''
1669
'''
1670
'''
1671
'''
1672
'''
1673
'''
1674
'''
1675
'''
1676
'''
1677
'''
1678
'''
1679
'''
1680
'''
1681
'''
1682
'''
1683
'''
1684
'''
1685
'''
1686
'''
1687
'''
1688
'''
1689
'''
1690
'''
1691
'''
1692
'''
1693
'''
1694
'''
1695
'''
1696
'''
1697
'''
1698
'''
1699
'''
1700
'''
1701
'''
1702
'''
1703
'''
1704
'''
1705
'''
1706
'''
1707
'''
1708
'''
1709
'''
1710
'''
1711
'''
1712
'''
1713
'''
1714
'''
1715
'''
1716
'''
1717
'''
1718
'''
1719
'''
1720
'''
1721
'''
1722
'''
1723
'''
1724
'''
1725
'''
1726
'''
1727
'''
1728
'''
1729
'''
1730
'''
1731
'''
1732
'''
1733
'''
1734
'''
1735
'''
1736
'''
1737
'''
1738
'''
1739
'''
1740
'''
1741
'''
1742
'''
1743
'''
1744
'''
1745
'''
1746
'''
1747
'''
1748
'''
1749
'''
1750
'''
1751
'''
1752
'''
1753
'''
1754
'''
1755
'''
1756
'''
1757
'''
1758
'''
1759
'''
1760
'''
1761
'''
1762
'''
1763
'''
1764
'''
1765
'''
1766
'''
1767
'''
1768
'''
1769
'''
1770
'''
1771
'''
1772
'''
1773
'''
1774
'''
1775
'''
1776
'''
1777
'''
1778
'''
1779
'''
1780
'''
1781
'''
1782
'''
1783
'''
1784
'''
1785
'''
1786
'''
1787
'''
1788
'''
1789
'''
1790
'''
1791
'''
1792
'''
1793
'''
1794
'''
1795
'''
1796
'''
1797
'''
1798
'''
1799
'''
1800
'''
1801
'''
1802
'''
1803
'''
1804
'''
1805
'''
1806
'''
1807
'''
1808
'''
1809
'''
1810
'''
1811
'''
1812
'''
1813
'''
1814
'''
1815
'''
1816
'''
181
```

```

=====
USER DATA COLLECTION WITH PRIVACY SAFEGUARDS
=====

Enter your name: Yashwanth
Enter your age: 23
Enter your email: pateruyashwanth6671@gmail.com

=====
DATA COLLECTION SUMMARY
=====

Original Data Collected:
  Name: Yashwanth
  Age: 23
  Email: pateruyashwanth6671@gmail.com

Anonymized/Safeguarded Data:
  Anonymous ID (name hash): f49876d80754ccfc
  Age Range (anonymized): 18-25
  Email Hash (hashed): 72503dcb95c46f52

Data prepared for secure storage:
  Hash ID (for tracking): 8014ff3e137fc3c6
  Timestamp: 2026-01-22T13:39:24.414973

=====
PRIVACY NOTES:
=====

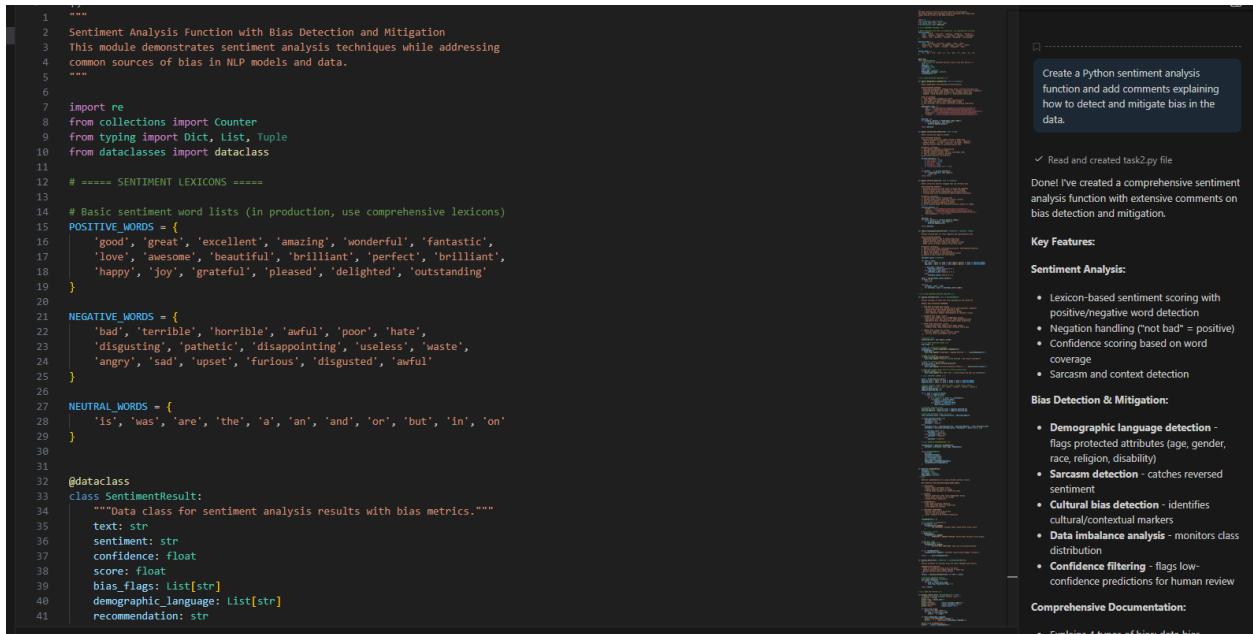
  ✓ Original data should be encrypted before storage
  ✓ Anonymized data can be used for analytics safely
  ✓ Implement role-based access controls
  ✓ Keep audit logs of all data access
  ✓ Regularly review data retention policies
  ✓ Comply with GDPR, CCPA, and local privacy laws

```

Task Description #2:

- Ask Copilot to generate a Python function for sentiment analysis.

Then prompt Copilot to identify and handle potential biases in the data.



Create a Python sentiment analysis function and add comments explaining how to detect and mitigate bias in the data.

✓ Read and created task2.py file

Don't I've created a comprehensive sentiment analysis function with extensive comments on bias detection and mitigation.

Key Features:

Sentiment Analysis:

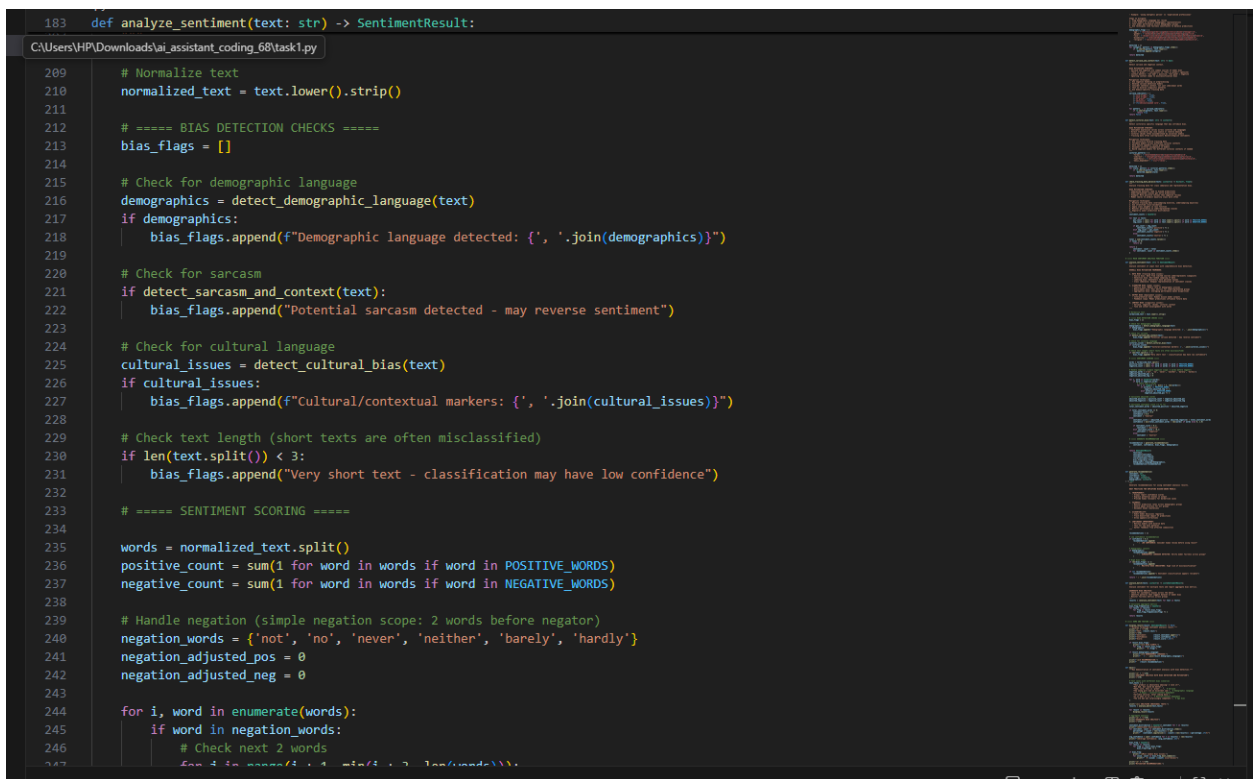
- Lexicon-based sentiment scoring with positive/negative word detection
- Negation handling ("not bad" = positive)
- Confidence scoring based on word coverage
- Sarcasm and context detection

Bias Detection & Mitigation:

- **Demographic language detection** - flags protected attributes (age, gender, race, religion, disability)
- **Sarcasm detection** - catches reversed sentiment
- **Cultural bias detection** - identifies cultural/contextual markers
- **Data imbalance analysis** - monitors class distribution
- **Confidence filtering** - flags low-confidence predictions for human review

Comprehensive Documentation:

- Explains 4 types of bias: data bias



```

RECOMMENDATION:
  ▲ LOW CONFIDENCE: Consider human review before using result | ▲ DEMOGRAPHIC LANGUAGE DETECTED: Verify model fairness across groups

=====
AGGREGATE BIAS ANALYSIS
=====

Sentiment Distribution:
  Positive: 2/8 (25.0%)
  Neutral: 5/8 (62.5%)
  Negative: 1/8 (12.5%)

Average Confidence: 5.85%

Most Common Bias Issues:
  • Demographic language detected: age, gender (2 occurrences)
  • Cultural/contextual markers: hyperbolic (1 occurrences)
  • Cultural/contextual markers: emoji_dependent (1 occurrences)

=====
MITIGATION RECOMMENDATIONS:
=====

1. COLLECT DIVERSE DATA:
  - Include multiple languages, cultures, demographics
  - Balance sentiment classes
  - Ensure representation of all user groups

2. IMPROVE PREPROCESSING:
  - Better sarcasm and negation detection
  - Handle emojis and modern language
  - Normalize cultural variations

3. ROBUST EVALUATION:
  - Test across demographic groups
  - Use fairness metrics (group calibration, equalized odds)
  - Conduct user studies with diverse participants

4. ONGOING MONITORING:
  - Track prediction distribution over time
  - Detect feedback loops
  - Audit decisions regularly

5. TRANSPARENCY:
  - Report confidence scores
  - Explain model limitations
  - Allow human review for important decisions

```

Task Description #3:

- Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines like transparency and fairness.

```
1 """
2 Ethical AI Product Recommendation System
3 This module demonstrates best practices for building fair, transparent, and
4 user-respecting recommendation systems with ethical AI guidelines.
5 """
6
7 import json
8 import math
9 from datetime import datetime
10 from typing import List, Dict, Tuple, Optional
11 from dataclasses import dataclass, asdict, field
12 from collections import defaultdict
13
14
15 # ===== DATA STRUCTURES =====
16
17 @dataclass
18 class Product:
19     """Represents a product in the recommendation system."""
20     id: str
21     name: str
22     category: str
23     price: float
24     rating: float
25     vendor: str
26     features: Dict[str, str]
27
28
29 @dataclass
30 class UserProfile:
31     """
32     User profile for recommendations.
33
34     ETHICAL PRINCIPLE - USER AUTONOMY:
35     - Store only necessary user data
36     - Allow users to opt-out of tracking
37     - Enable data deletion and portability
38     - Be transparent about what data we collect
39     """
40     user_id: str
41     name: str
42     preferences: Dict[str, float] # category -> preference score (0-1)
43     budget_range: Tuple[float, float]
44     past_purchases: List[str] = field(default_factory=list)
45     opt_in_recommendations: bool = True
46     opt_in_tracking: bool = True
47     feedback_history: List['RecommendationFeedback'] = field(default_factory=list)
48
49
50 """
51 Additional code for the recommendation system, including data loading,
52 recommendation algorithms, and user interaction logic.
53 """
```

Create a Python product recommendation program and include ethical AI guidelines such as transparency, fairness, and user feedback in the code.

✓ Created Python file

Done! I've created a comprehensive ethical AI product recommendation system with extensive documentation on transparency, fairness, and user feedback.

Key Ethical Principles Implemented:

1. Transparency:

- Every recommendation includes detailed explanations for why it was suggested
- Confidence scores disclosed for each recommendation
- Potential biases clearly identified and explained
- Users understand the reasoning behind each suggestion

2. Fairness:

- Diversity constraints prevent filter bubbles (recommending only familiar categories)
- Diverse recommendations across categories and vendors
- Vendor monopoly prevention
- Price fairness (balance across price points, not just expensive items)

3. User Autonomy:

- Opt-in/opt-out controls for recommendations and tracking

task3.py

Describe what to build next

Agent Auto

```
PS C:\Users\HP\Downloads\ai_assistant_coding_68> & C:\Users\HP\AppData\Local\Programs\Python\Python311\python.exe c:/Users/HP/Downloads/ai_assistant_coding_68/task3.py

=====
ETHICAL AI PRODUCT RECOMMENDATION SYSTEM
=====

=====
RECOMMENDATIONS FOR ALICE
=====

#1: Premium Laptop
Price:      $1299.99
Rating:     4.8/5.0
Confidence: 95.0%
Diversity:  50.0%

✓ WHY THIS RECOMMENDATION:
  • You have strong interest in Electronics products
  • Highly rated (4.8/5 stars)
  • Includes popular features in this category

⚠ POTENTIAL BIASES:
  • Limited user history - recommendation may not be accurate
  • High-rated items may be over-recommended (popular item bias)

🔄 ALTERNATIVES:
  • Budget Laptop ($399.99)

=====
RECOMMENDATIONS FOR BOB
=====

#1: Running Shoes
Price:      $149.99
Rating:     4.6/5.0
Confidence: 82.5%
Diversity:  33.3%

✓ WHY THIS RECOMMENDATION:
  • You have strong interest in Sports products
  • Price ($149.99) fits your budget range
  • Highly rated (4.6/5 stars)
  • Includes popular features in this category

⚠ POTENTIAL BIASES:
  • Limited user history - recommendation may not be accurate
  • High-rated items may be over-recommended (popular item bias)

🔄 ALTERNATIVES:
  • Coffee Maker ($59.99)
  • Wireless Headphones ($89.99)
```

Task Description #4:

- Prompt Copilot to generate logging functionality in a Python web application. Then, ask it to ensure the logs do not record sensitive information.

```
1  """
2  Secure Logging Functionality for Python Web Applications
3  This module demonstrates best practices for logging in web applications while
4  protecting sensitive user information and maintaining privacy compliance.
5  """
6
7  import logging
8  import logging.handlers
9  import re
10 import json
11 from typing import Dict, Any, Optional
12 from datetime import datetime
13 from functools import wraps
14 import hashlib
15
16
17 # ===== SENSITIVE DATA PATTERNS =====
18
19 # Patterns to detect and redact sensitive information
20 SENSITIVE_PATTERNS = {
21     # Credit card patterns (simplified)
22     'credit_card': r'\b\d{4}[\s-]?d{4}[\s-]?d{4}[\s-]?d{4}\b',
23
24     # Email addresses
25     'email': r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b',
26
27     # Phone numbers
28     'phone': r'\b(?:\+?1[-.\s]?)(?:[0-9]{3})?(?:[-.\s]?[0-9]{3})?(?:[-.\s]?[0-9]{4})\b',
29
30     # Social Security Numbers (US)
31     'ssn': r'\b(?:[0-9]{3})-[0-9]{2}-[0-9]{4}\b',
32
33     # API keys and tokens
34     'api_key': r'[Aa]pi[_-]?[Kk]ey["\']?\s*[:=]\s*["\']?[A-Za-z0-9]{20,}\b',
35
36     # Passwords in common formats
37     'password': r'^(?i)(password|passwd|pwd)["\']?\s*[:=]\s*["\']?[^\\s\'"]{8,}$',
38
39     # Bearer tokens
40     'bearer_token': r'[Bb]earer\s+[A-Za-z0-9._-]+',
41
42     # Database connection strings
43     'db_connection': r'^(?i)(user|password|host)=([^\s&]+)',
44
45     # IPv4 addresses (less sensitive but can be PII)
46     'ipv4': r'\b(?:?:25[0-5]|2[0-4][0-9]|01[0-9]?[0-9]?[0-9])\.(?:?:25[0-5]|2[0-4][0-9]|01[0-9]?[0-9]?[0-9])\b',
47 }
```

Create logging functionality in a Python web app and ensure logs do not capture sensitive user information.

✓ Created Python file

Done! I've created a comprehensive secure logging system for Python web applications that automatically protects sensitive user information.

Key Features:

- Pattern matching for credit cards, emails, phone numbers, SSNs, API keys, passwords, tokens, etc.
- Dictionary field detection (redacts values of fields named "password", "email", etc.)
- Automatic hashing of PII for tracking without exposing identity
- Consistent [REDACTED] placeholders

Logging Configuration:

- Console handler for development/monitoring
- Rotating file handler (10MB auto-rotation, keeps 10 backups)
- Optional syslog handler for centralized logging
- Secure formatter that automatically redacts all logs
- Configurable log levels and retention

Privacy-Aware Logging Functions:

- `log_user_login()` - Track logins without

task4.py

Describe what to build next.

```

PS C:\Users\HP\Downloads\ai_assistant_coding_68> & C:\Users\HP\AppData\Local\Programs\Python\Python311\python.exe c:/Users/HP/Downloads/ai_assistant_coding_68/task4.py

=====
SECURE LOGGING FOR PYTHON WEB APPLICATIONS
=====

📄 LOGGING SCENARIOS:
=====

1 USER LOGIN LOGGING:
[2026-01-22 14:00:08,862] INFO - web_app - User HASH:f9e8e37d2e825eb0 logged in successfully
[2026-01-22 14:00:08,864] WARNING - web_app - Failed login attempt for user HASH:f9e8e37d2e825eb0
    ✓ Logged (sensitive email hashed)

2 API REQUEST LOGGING:
[2026-01-22 14:00:08,865] INFO - web_app - API GET /api/users/profile by HASH:f9e8e37d2e825eb0
    ✓ Logged (user ID hashed)

3 DATA ACCESS LOGGING:
[2026-01-22 14:00:08,866] INFO - web_app - User HASH:f9e8e37d2e825eb0 performed READ on payment_records
    ✓ Logged (sensitive access tracked)

4 ERROR LOGGING WITH CONTEXT:
[2026-01-22 14:00:08,867] ERROR - web_app - Error for user HASH:4e920dc577a96695: Payment processing failed
    ✓ Logged (sensitive fields automatically redacted)

5 SECURITY EVENT LOGGING:
[2026-01-22 14:00:08,868] ERROR - web_app - SECURITY EVENT [BRUTE_FORCE_ATTEMPT]: Multiple failed login attempts from IP [REDACTED]
    ✓ Logged (security incident tracked)

6 SENSITIVE DATA REDACTION EXAMPLES:

Original: User payment card 4532-1234-5678-9010 was processed
Redacted: User payment card [REDACTED] was processed

Original: API key: sk_live_51234567890abcdef was used
Redacted: API key: sk_live_51234567890abcdef was used

Original: Password reset for user@example.com successful
Redacted: Password reset for [REDACTED] successful

Original: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 authenticated
Redacted: [REDACTED] authenticated

Original: SSN 123-45-6789 verified
Redacted: SSN [REDACTED] verified

Original: Contact: +1-800-555-0123 or john.doe@company.com
Redacted: Contact: +[REDACTED] or [REDACTED]

```

Task Description #5:

- Ask Copilot to generate a machine learning model. Then, prompt it to add documentation on how to use the model responsibly (e.g., explainability, accuracy limits).

taskSpy > ...

C:\Users\HPA\Downloads\ai_assistant_coding_68\secure_app.log

```
225
226
227
228 @dataclass
229 class ApplicantProfile:
230     """Loan applicant profile for demonstration."""
231     applicant_id: str
232     age: float
233     income: float
234     credit_score: float
235     employment_years: float
236     protected_group: str # For fairness analysis
237
238
239 @dataclass
240 class PredictionResult:
241     """
242     ML prediction result with full transparency.
243
244     TRANSPARENCY PRINCIPLE:
245     - Every prediction includes explanation
246     - Show confidence/probability
247     - Disclose factors contributing to decision
248     - Include fairness assessment
249     - Document model limitations
250     """
251     applicant_id: str
252     prediction: int # 0: Denied, 1: Approved
253     probability: float # Confidence score 0-1
254     explanation: List[str] # Why this decision
255     fairness_flag: bool # Potential fairness concern
256     fairness_reason: Optional[str]
257     appeal_available: bool # Can applicant appeal
258     model_version: str
259
260
261 @dataclass
262 class FairnessMetrics:
263     """Comprehensive fairness metrics."""
264     overall_accuracy: float
265     group_accuracies: Dict[str, float]
266     approval_rates: Dict[str, float] # Demographic parity
267     false_negative_rates: Dict[str, float] # Equalized odds
268     false_positive_rates: Dict[str, float] # Equalized odds
269     demographic_parity_difference: float # Max difference in approval rates
270     equalized_odds_difference: float # Max difference in error rates
```

PYTHON SCRIPT FOR USER DATA COLLE...

Create a machine learning model in Python and document how to use it responsibly, including limitations and fairness considerations.

✓ Created Python file

Done! I've created a comprehensive responsible ML model with extensive documentation on fairness and limitations.

Key Features:

Model Card (Comprehensive Documentation):

- Clear intended use and limitations
- Critical warnings about fairness concerns
- Known biases and mitigation strategies
- Regulatory considerations (Fair Housing Act, GDPR, CCPA, etc.)
- Deployment requirements and monitoring

Fairness Metrics:

- Demographic Parity (approval rates across groups)
- Equalized Odds (false positive/negative rates)
- Disparate Impact analysis
- Group-level accuracy tracking
- Automatic fairness alert flags

Transparency Features:

- Human-readable explanations for every prediction
- Confidence scores disclosed
- Fairness concerns flagged for human review
- Rights information (appeals, transparency,

taskSpy

Describe what to build next


```
PS C:\Users\HP\Downloads\ai_assistant_coding_68> ^C
PS C:\Users\HP\Downloads\ai_assistant_coding_68> C:/Users/HP/Downloads/ai_assistant_coding_68/.venv/Scripts/python.exe C:\Users\HP\Downloads\ai_assistant_coding_68\task5.py
```

=====

RESPONSIBLE MACHINE LEARNING MODEL

=====

LOAN ELIGIBILITY MODEL CARD

MODEL OVERVIEW:

Name: Loan Eligibility Classifier v1.0
Type: Binary Classification (RandomForestClassifier)
Training Date: 2026-01-22
Purpose: Predict loan eligibility for demonstration purposes
Intended Use: DEMONSTRATION ONLY - Not for production lending decisions

INTENDED USE:

✓ DO USE FOR:

- Educational demonstrations
- Understanding ML fairness concepts
- Testing and validation workflows
- Fairness auditing techniques

✗ DO NOT USE FOR:

- Actual lending decisions
- Production financial services
- High-stakes decisions affecting individuals
- Autonomous decision-making without human review

CRITICAL LIMITATIONS:

1. BIASED DATA:

- Training data contains historical lending patterns
- Reflects past discrimination and biases
- May perpetuate unfair decisions

2. INCOMPLETE INFORMATION:

- Only uses demographic and income features
- Missing important factors (credit history, employment stability)
- Cannot account for life circumstances

3. MODEL LIMITATIONS:

- Assumes historical patterns predict future outcomes
- Cannot capture economic changes or individual circumstances
- Oversimplifies complex financial decisions

4. FAIRNESS CONCERNS:

- Model may have disparate impact on protected groups